

Symbolic Execution: Application

1

Outline

- Concrete Execution
- Symbolic Execution Theory
 - How to execute symbolically
 - Min function example
 - Negative function exercise
 - Limitations
- **Symbolic Execution Tools**
 - **Binary Search function example**
 - **Discussion**

2

Symbolic Execution Platforms

Name	Platform	Languages	Where?
Pex / Intellitest	.Net	CLR	Visual Studio 2017 RC / www.pexforfun.com
KLEE	Clang	LLVM	klee.github.io
Pathfinder	Java	JVM	javapathfinder.sourceforge.net

Let's try one out on a real piece of code.

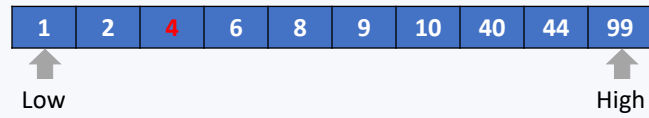
3

Binary Search

What is Binary Search?

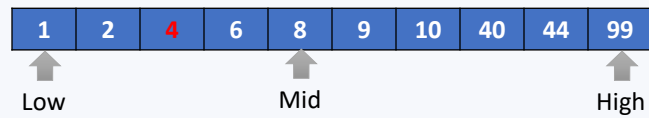
4

Binary Search



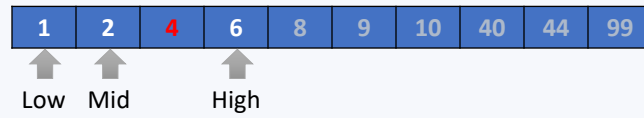
5

Binary Search



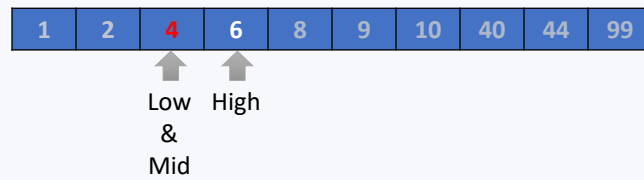
6

Binary Search



7

Binary Search



8

Binary Search



“Although the basic idea of binary search is comparatively straightforward, the details can be surprisingly tricky...” – Donald Knuth [2]

9

Brief History of Binary Search

- **1946:** First mention of Binary Search [5]
- **1960:** First Binary Search not limited to input lengths one less than a power of 2 [5]
- **1960-1986:** Lots and *lots* of implementations...
- **1986:** Publication of a “proven” implementation [1]
- **2006:** The Java JDK *still* contains a bug in the Binary Search function! [3, 6]

Can you see what is wrong in the Binary Search implementation?

10

Binary Search

```
static int BinarySearch(int key, Func<int,int> f, int low, int high) {
    while(low <= high) {
        int mid = (low + high) / 2;

        if(f(mid) > key) {
            high = mid - 1;
        } else if(f(mid) < key) {
            low = mid + 1;
        } else {
            return mid; // Key Value Found!
        }
    }

    return -1; // Key Value NOT Found.
}
```

11

Symbolic Execution on Binary Search via Pex

First, limit the inputs to match our assumptions:

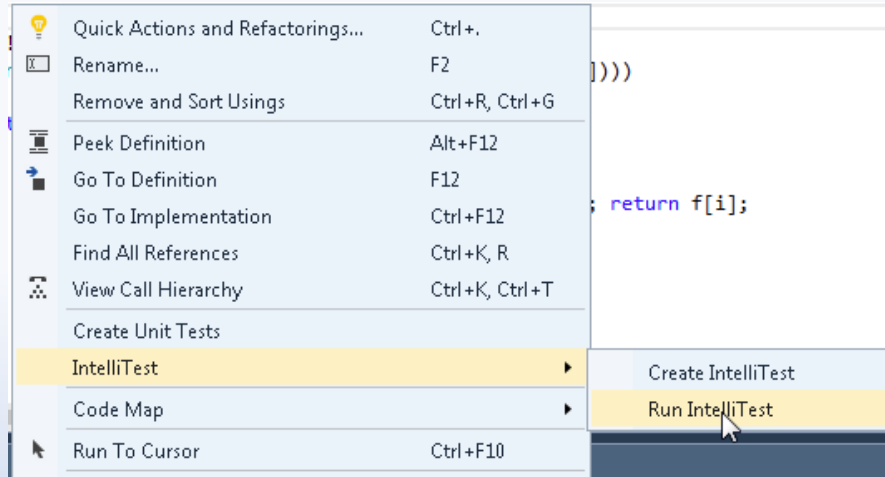
```
static int BinarySearchArrayTestWrapper(int key, int[] f)
{
    if (f != null && f.Length > 0 &&
        Contract.ForAll(0, f.Length - 1, i => f[i] < f[i + 1]))
    {
        return BinarySearch(key,
            delegate (int i)
            {
                Debug.Assert(i >= 0 && i < f.Length); return f[i];
            },
            0, f.Length-1);
    }

    return -1;
}
```

12

Symbolic Execution on Binary Search via Pex

Right click in the function body, and *Run IntelliTest*:



13

Symbolic Execution on Binary Search via Pex

Tests are automatically generated:

✓ 15 ✗ 0 43/44 blocks, 1/1 asserts, 134 runs				
	key	f	result	Summary / Exception
1	0	null	-1	
2	0	{}	-1	
3	0	{0}	0	
4	512	{1}	-1	
5	int.MaxValue	{int.MaxValue}	0	
6	513	{10}	-1	
7	0	{1}	-1	
8	0	{0, 0}	-1	
9	int.MinValue	{-2147483648}	-1	
10	0	{0, 1}	0	
11	1008	{1001, 1003}	-1	
12	0	{0, 1, 0}	-1	
13	9	{0, 8, 9}	2	
14	0	{1, 8, 9}	-1	
15	-1073741824	{-2147483648}	-1	

14

Symbolic Execution on Binary Search via Pex

We didn't find anything!

Does that mean the implementation is correct?

15

Symbolic Execution on Binary Search via Pex

First, limit the inputs to match our assumptions:

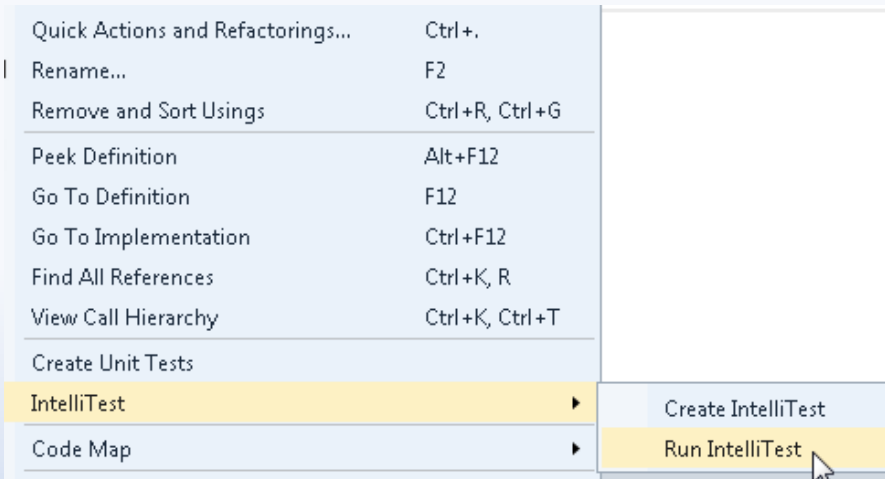
```
public static int BinarySearchFuncTestWrapper(int key, int size)
{
    if (size > 0)
    {
        return BinarySearch(key, delegate (int i)
        {
            Debug.Assert(i >= 0 && i <= size); return i;
        }, 0, size);
    }

    return -1;
}
```

16

Symbolic Execution on Binary Search via Pex

Right click in the function body, and *Run IntelliTest*:



17

Symbolic Execution on Binary Search via Pex

Tests are automatically generated:

✓ 8 ✗ 1 38/38 blocks, 1/1 asserts, 95 runs					
	▲	key	size	result	Summary / Exception
✓	1	0	0	-1	
✓	2	0	1	0	
✓	3	int.MinValue	1	-1	
✓	4	1	1	1	
✓	5	0	2	0	
✓	6	0	877	0	
✓	7	0	int.MaxValue	0	
✗	8	2147483646	int.MaxValue		TraceAssertionException
✓	9	2	1	-1	
✓	10	805306368	int.MaxValue		path bounds exceeded

18

Symbolic Execution on Binary Search via Pex

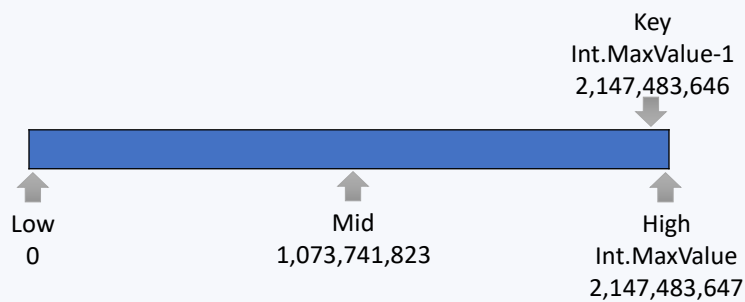
Tests are automatically generated:

Details

```
[TestMethod]
[PexGeneratedBy(typeof(ProgramTest))]
[PexRaisedException(typeof(
TraceAssertionException))]
public void
BinarySearchFuncTestWrapperThrowsTraceAssertionE
xception441()
{
    using (PexTraceListenerContext.Create())
    {
        int i;
        i = this.BinarySearchFuncTestWrapper(2147483646,
int.MaxValue);
    }
}
```

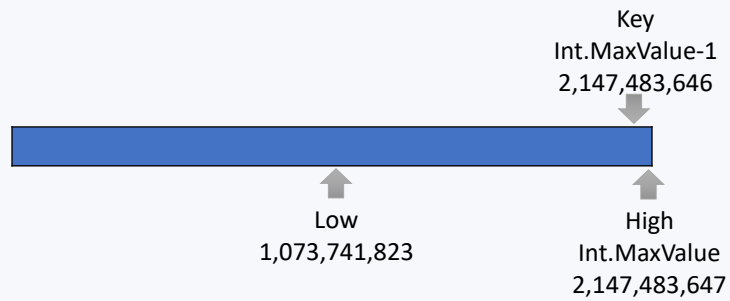
19

Binary Search Bug



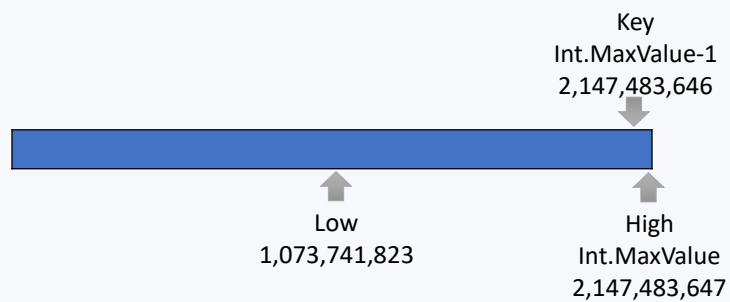
20

Binary Search Bug



21

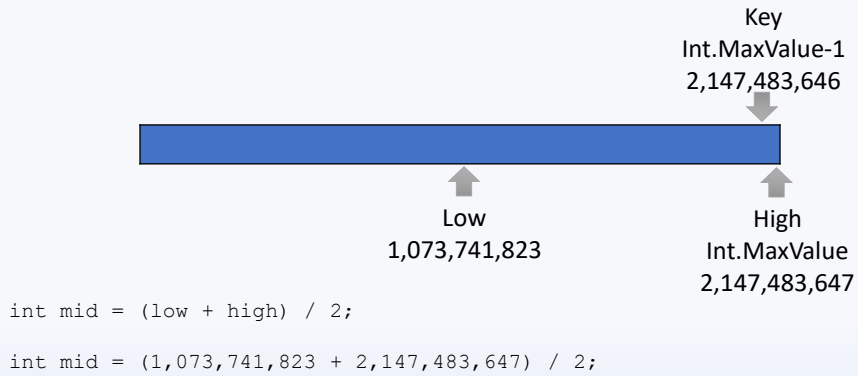
Binary Search Bug



```
int mid = (low + high) / 2;
```

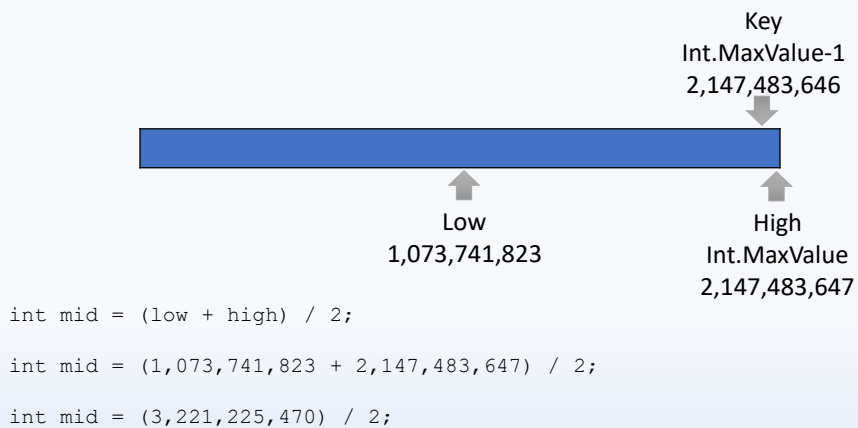
22

Binary Search Bug



23

Binary Search Bug



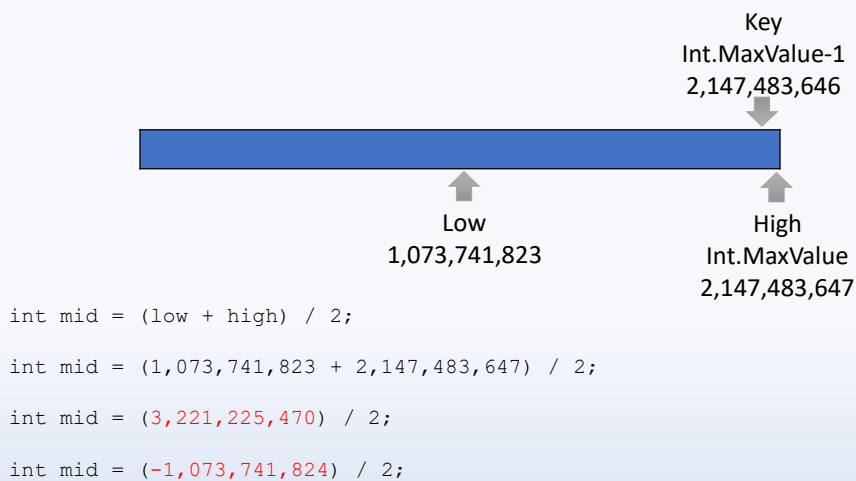
24

Binary Search Bug



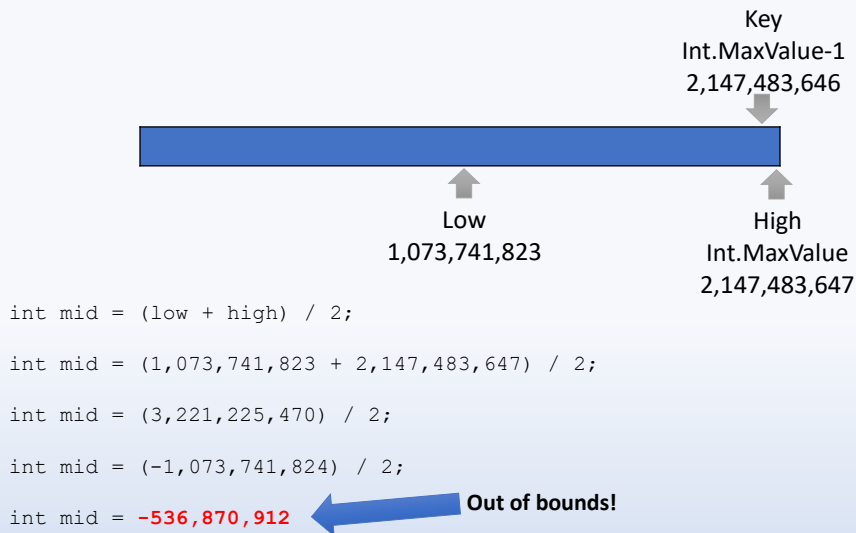
25

Binary Search Bug



26

Binary Search Bug



27

Binary Search Bug

```
static int BinarySearch(int key, Func<int,int> f, int low, int high) {
    while(low <= high) {
        int mid = (low + high) / 2;

        if(f(mid) > key) {
            high = mid - 1;
        } else if(f(mid) < key) {
            low = mid + 1;
        } else {
            return mid; // Key Value Found!
        }
    }

    return -1; // Key Value NOT Found.
}
```

28

Binary Search

```
static int BinarySearch(int key, Func<int,int> f, int low, int high) {
    while(low <= high) {
        int mid = low + (high - low) / 2;

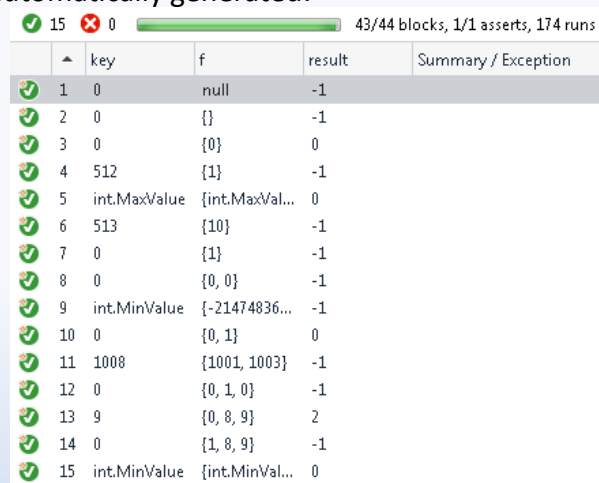
        if(f(mid) > key) {
            high = mid - 1;
        } else if(f(mid) < key) {
            low = mid + 1;
        } else {
            return mid; // Key Value Found!
        }
    }

    return -1; // Key Value NOT Found.
}
```

29

Symbolic Execution on Binary Search via Pex

Tests are automatically generated:



	▲	key	f	result	Summary / Exception
✓ 15	✗ 0				43/44 blocks, 1/1 asserts, 174 runs
✓ 1		0	null	-1	
✓ 2		0	{}	-1	
✓ 3		0	{0}	0	
✓ 4		512	{1}	-1	
✓ 5		int.MaxValue	{int.MaxValue...	0	
✓ 6		513	{10}	-1	
✓ 7		0	{1}	-1	
✓ 8		0	{0, 0}	-1	
✓ 9		int.MinValue	{-21474836...	-1	
✓ 10		0	{0, 1}	0	
✓ 11		1008	{1001, 1003}	-1	
✓ 12		0	{0, 1, 0}	-1	
✓ 13		9	{0, 8, 9}	2	
✓ 14		0	{1, 8, 9}	-1	
✓ 15		int.MinValue	{int.MinValue...	0	

30

Symbolic Execution on Binary Search via Pex

Tests are automatically generated:

 9
  0
  37/38 blocks, 1/1 asserts, 71 runs

	▲	key	size	result	Summary / Exception
	1	0	0	-1	
	2	0	1	0	
	3	int.MinValue	1	-1	
	4	1	1	1	
	5	0	2	0	
	6	0	877	0	
	7	0	int.MaxValue	0	
	8	2130706431	int.MaxValue	2130706431	
	9	2	3	2	
	10	1342177280	int.MaxValue		path bounds exceeded

31

Discussion

- Why were previously “proven” implementations incorrect?
- Why didn’t the array version catch the overflow?
- Is symbolic execution *really* exhaustive?
- When does it make sense to use symbolic execution?

32

References

1. Bentley, Jon. *Programming pearls*. Addison-Wesley Professional, 1986.
2. Knuth, Donald Ervin. *The art of computer programming: sorting and searching*. Vol. 3. Pearson Education, 1998.
3. Ruggieri, Salvatore. "On computing the semi-sum of two integers." *Information processing letters* 87.2 (2003): 67-71.
4. <https://research.googleblog.com/2006/06/extra-extra-read-all-about-it-nearly.html>
5. https://en.wikipedia.org/wiki/Binary_search_algorithm
6. http://bugs.java.com/bugdatabase/view_bug.do?bug_id=5045582